

실전반 1회 · 첫 데이터가 흐르는 시스템

- [이번 회차에서 할 일](#)
- [1. 시작 전 준비 \(복습·환경 확인\)](#)
- [2. 핵심 개념](#)
- [3. 활동 1 — 완성 시스템 시연과 아키텍처 읽기 · 50분](#)
- [4. 활동 2 — 세 센서 CSV를 공통 관측 형식으로 매핑 · 50분](#)
- [5. 활동 3 — 제공 적재 함수를 PostgreSQL에 연결 · 50분](#)
- [6. 활동 4 — 최근 60초를 조회하고 원본 사이클 추적 · 50분](#)
- [7. 실습 과제](#)
- [8. 완료 확인 체크리스트](#)
- [9. 회고 \(10분\)](#)
- [강사 노트](#)
- [용어](#)

날짜·시간	2026-10-31 (토) 14:00~18:00	모듈	M1 · 원 일정 3번
블록	구현 B1 · B2 · B3	산출물	B1→B2→B3 데이터 경로
완료 확인	단위·설비 ID·원본/합성 구분을 포함한 관측 조회		

회차별 블록 로드맵 — 하나의 시스템에 매 회차 기능을 더한다

구현 새로 만든다 · **확장** 조건·연결을 바꾼다 · **플랫폼** 교육용 플랫폼에 연결한다

실전반 4학년 · 9회 × 4시간 (합수·질의·정책 수정과 검증)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 10/31	구현	구현	구현						
2회 11/07		확장	확장						
3회 11/21			확장	구현					
4회 11/28		확장	확장		구현			구현	
5회 01/25				확장		구현	구현		
6회 01/26			플랫폼	확장+P			확장		
7회 01/27						확장+P	확장+P	구현+P	
8회 01/28	확장				확장			확장	
9회 01/29									확장+P

통합반 2·3학년 · 10회 × 4시간 (제공 코드의 매핑·설정·조건 완성)

	B1	B2	B3	B4	B5	B6	B7	B8	B9
1회 11/14	구현	구현							
2회 01/25		확장	구현						
3회 01/26				구현					
4회 01/27		확장	확장		구현			구현	
5회 01/28				확장		구현			
6회 01/29							구현	구현	
7회 02/01			플랫폼	확장+P					
8회 02/02						확장+P	확장+P	확장+P	
9회 02/03									확장+P
10회 02/04	확장				확장			확장	확장

이번 회차가 다루는 블록

이번 회차에서 할 일

이번 학기에 만드는 시스템은 유압설비의 냉각 이상을 발견하고, 설비 관계와 SOP 를 확인한 뒤, 사람이 승인한 조치를 실행하고 새 관측으로 회복을 다시 측정한다. 오늘은 그 맨 앞, Data Sources 계층의 B1(데이터)·B2(품질 검사)·B3(PostgreSQL)을 잇는다. UCI 유압설비 데이터의 세 센서 파일은 샘플링 주기와 단위가 모두 다르다. 이것을 하나의 공통 관측 레코드로 매핑하고, 제공 품질 검사기와 적재 함수로 PostgreSQL 에 넣은 뒤, 최근 60초를 조회해 그 값이 어느 원본 사이클에서 왔는지 추적한다. 회차가 끝나면 "이 값은 HYD-01 의 TS1, 단위 °C, UCI 원본 사이클 102 의 12초째, 합성 아님"이라고 DB 조회 결과로 말할 수 있어야 한다.

학습 목표

- 완성 시스템 화면에서 B1~B9 블록과 네 계층(Data Sources / Ontology / Agents / Orchestration)의 위치를 짚는다.
- 1 Hz·10 Hz·100 Hz 센서를 1초 mean·min·max·n 으로 축약하는 함수를 고치고, 강사 제공 축약본과 값이 같은지 검증한다.
- 관측 레코드에 설비 ID·단위·원본 사이클·위상(elapsed_s)·원본/합성 구분을 넣어 PostgreSQL 에 적재한다.
- run_id 조건이 있는 최근 60초 조회와 없는 조회의 결과 차이를 설명한다.

1. 시작 전 준비 (복습·환경 확인)

모든 명령은 `lecture/system` 에서 실행한다.

```
cd lecture/system
docker compose ps --format "table {{.Name}}\t{{.Status}}"
```

예상 출력 (상태 시간은 다르다):

NAME	STATUS
hydops-neo4j	Up About an hour
hydops-postgres	Up About an hour

원본 데이터 파일은 사이클 하나가 한 줄이다. 줄 수와 한 줄의 열 수를 확인한다.

```

wc -l data/raw/TS1.txt data/raw/PS1.txt data/raw/FS1.txt data/raw/profile.txt
awk -F'\t' 'FNR==1{print FILENAME, NF}' data/raw/TS1.txt data/raw/FS1.txt
data/raw/PS1.txt data/raw/profile.txt

```

```

2205 data/raw/TS1.txt
2205 data/raw/PS1.txt
2205 data/raw/FS1.txt
2205 data/raw/profile.txt
8820 total
data/raw/TS1.txt 60
data/raw/FS1.txt 600
data/raw/PS1.txt 6000
data/raw/profile.txt 5

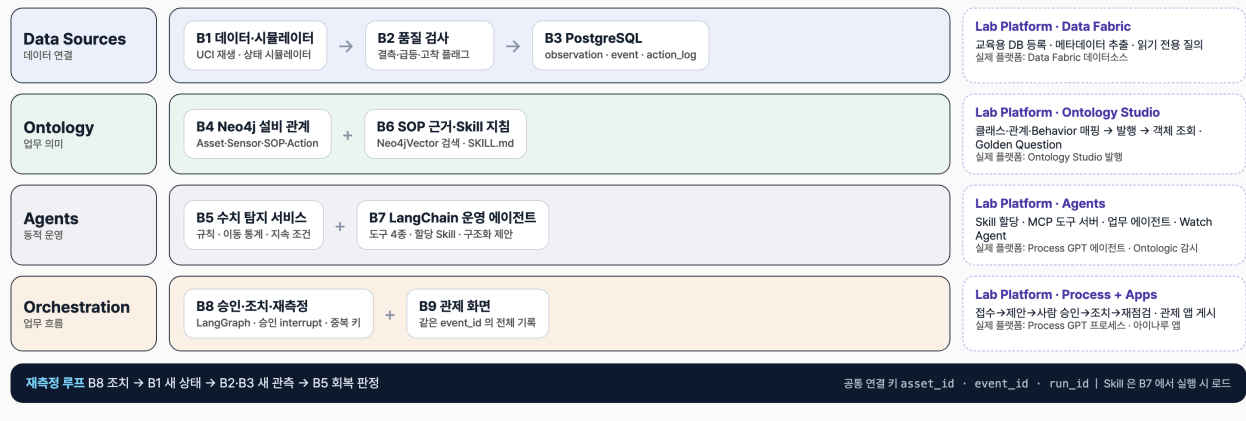
```

2,205 사이클 × 60초이고, 한 줄의 열 수가 60(TS1, 1 Hz) · 600(FS1, 10 Hz) · 6000(PS1, 100 Hz)으로 다르다. `profile.txt` 의 5열은 냉각기 상태·밸브·펌프 누설·어류물레이터·안정 플래그로, **정답 라벨**이다.

2. 핵심 개념

완성 시스템 아키텍처 — 교육용 블록 B1~B9 와 플랫폼 계층

수업의 기준 구조: Data Sources → Ontology → Agents → Orchestration. 왼쪽은 초반(로컬 구현), 오른쪽은 후반(교육용 플랫폼 연결).



완성 시스템 아키텍처

오늘 다루는 것은 그림 맨 윗줄 Data Sources 의 B1 → B2 → B3 이다. 뒤 회차의 모든 블록은 이 줄이 만든 관측 레코드를 읽는다. 공통 연결 키는 그림 아래의 `asset_id` · `event_id` · `run_id` 인데, 오늘은 `asset_id` 와 `run_id` 를 처음 쓴다.

2.1 공통 관측 레코드

hydops/b1_data/uci.py 의 cycle_observations 가 만드는 레코드가 이 시스템의 표준 형식이다.

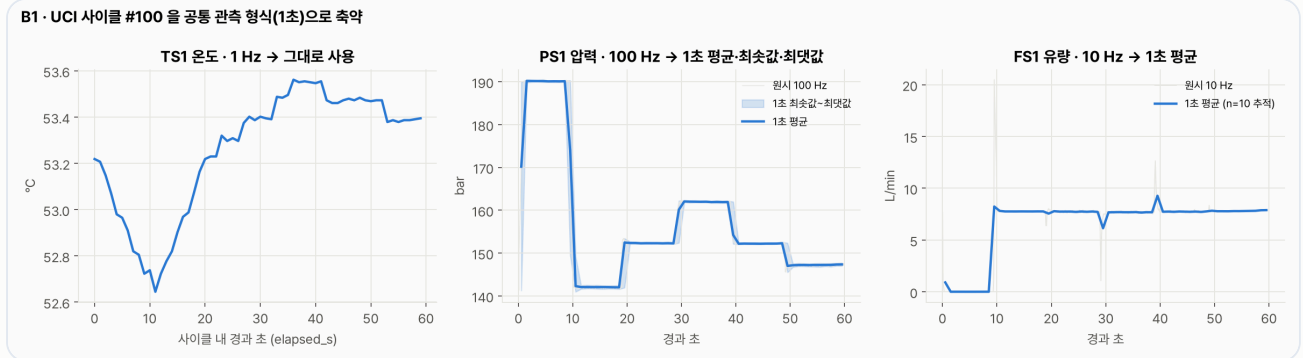
```
# lecture/system/hydops/b1_data/uci.py (발췌)
rows.append(
    {
        "asset_id": asset_id,
        "sensor_id": sid,
        "ts": replay_start + timedelta(seconds=s),
        "elapsed_s": s,
        "origin_cycle_id": int(cycle_id),
        "raw_value": mean,
        "unit": meta["unit"],
        "agg": None
        if meta["hz"] == 1
        else {
            "min": round(float(stats["min"][cycle_id, s]), 4),
            "max": round(float(stats["max"][cycle_id, s]), 4),
            "n": int(stats["n"][cycle_id, s]),
            "source_hz": meta["hz"],
        },
        "is_synthetic": False,
    }
)
```

필드	의미	주의
ts	재생 시각	실제 수집 시각이 아니다
elapsed_s	사이클 안의 경과 초(위상)	0~59
origin_cycle_id	UCI 원본 사이클 번호	시뮬레이터(합성)는 NULL
raw_value	1초 평균(원시 값)	결측이면 NULL
unit	°C / bar / L/min	센서마다 다르다
agg	1초 축약 통계 min·max·n·source_hz	1 Hz 센서는 없음
is_synthetic	원본/합성 구분	UCI 재생은 false

주의 — 재생 시각을 실제 수집 시각이라 부르지 않는다. UCI 데이터에는 실측 날짜가 없고, 우리가 정한 재생 시작 시각에 사이클을 이어 붙였을 뿐이다. 그래서 원본을 되찾을 수 있도록

origin_cycle_id 와 elapsed_s 를 따로 둔다.

2.2 1초 축약



B1 1초 축약

사이클 #100 의 세 센서다. TS1 은 이미 1초 간격이라 그대로 쓴다. PS1 은 1초에 100개 샘플이 있어 1초 안에서 크게 흔들리므로 평균만 남기면 순간 변동이 사라진다. 그래서 최솟값·최댓값 띠를 함께 남긴다. FS1 은 10개 샘플의 평균을 쓰고 원본 샘플 수 n=10 을 기록한다. 첫 샘플만 골라 쓰는 **썩아내기**는 축약이 아니다 — 활동 2 에서 그 차이를 숫자로 확인한다.

2.3 저장소 나누기

저장소를 나누는 기준 — DB 는 PostgreSQL 과 Neo4j 두 개

원시 파일은 파일 저장소, 수치 관측과 실행 이력은 PostgreSQL, 설비 관계와 SOP 검색은 Neo4j. 처음부터 시계열 전체를 그래프 노드로 만들지 않는다.

파일 저장소

data/raw

TS1.txt (1 Hz)
PS1.txt (100 Hz)
FS1.txt (10 Hz)
profile.txt (정답 라벨)

data/reduced

hydraulic_1s.npz
1초 mean/min/max/n

b6_sop/docs

SOP-*.md (frontmatter)

PostgreSQL · 수치와 이력

observation

run_id asset_id sensor_id ts(replay_ts) elapsed_s origin_cycle_id
raw_value value unit quality_flag agg is_synthetic

event

event_id run_id asset_id event_type window_start/end rule_version
evidence status proposal process_ref

approval · action_log

approval_id approver decision approved_value · idempotency_key
command_status sim_ts

verification · event_history

window(초치 이후) outcome metrics · 모든 상태 전이와 행위자

Neo4j · 관계와 문서

(Asset:Resource)

asset_id name cooling line

-[:HAS_SENSOR]→(Sensor:Measure)

sensor_id code quantity unit hz aliases

(SOP)-[:APPLIES_TO]→(Asset)

doc_id version effective_date status · -[:HAS_SECTION]→(SOPSection
{text, embedding}) · -[:ALLOWS]→(Action)

(Event 참조)-[:ON_ASSET]→·-[:CITES]→(SOPSection)

event_id status — 수치 상세는 PostgreSQL 에서

결합 기준 asset_id 로 그래프 맥락과 시계열 결과를 잇고, event_id 로 감지·근거·승인·조치 재추정을 잇는다. 플랫폼에서는 수치를 원천 DB 에 두고(가상 연결) 문서 지식만 그래프에 둔다.

저장소 분리

원시 파일은 파일 저장소, 수치 관측과 실행 이력은 PostgreSQL, 설비 관계와 SOP 는 Neo4j 에 둔다. 오늘은 가운데 **observation** 과 **run** 만 쓴다. **run.source** 는 **'uci_replay'** 또는 **'simulator'** 만 허용하는 CHECK 제약이 걸려 있다.

```

-- lecture/system/sql/01_schema.sql (발췌)
CREATE TABLE IF NOT EXISTS run (
    run_id          TEXT PRIMARY KEY,
    source          TEXT NOT NULL CHECK (source IN ('uci_replay', 'simulator')), -
    - 원본/합성 구분
    ...
);
CREATE TABLE IF NOT EXISTS observation (
    ...
    ts              TIMESTAMPTZ NOT NULL,          -- replay_ts (재생 시각, 실제 수집 시각
    - 아님)
    elapsed_s       INTEGER,                      -- 사이클 내 경과 초 (위상)
    origin_cycle_id INTEGER,                      -- UCI 원본 사이클 번호 (합성은 NULL)
    raw_value       DOUBLE PRECISION,             -- 원시 값 (결측이면 NULL)
    value           DOUBLE PRECISION,             -- 정제 값 (품질 불량이면 NULL)
    unit            TEXT NOT NULL,
    quality_flag    TEXT NOT NULL DEFAULT 'OK',
    agg             JSONB,                        -- 1초 축약 통계 (min/max/n)
    is_synthetic    BOOLEAN NOT NULL DEFAULT false
);

```

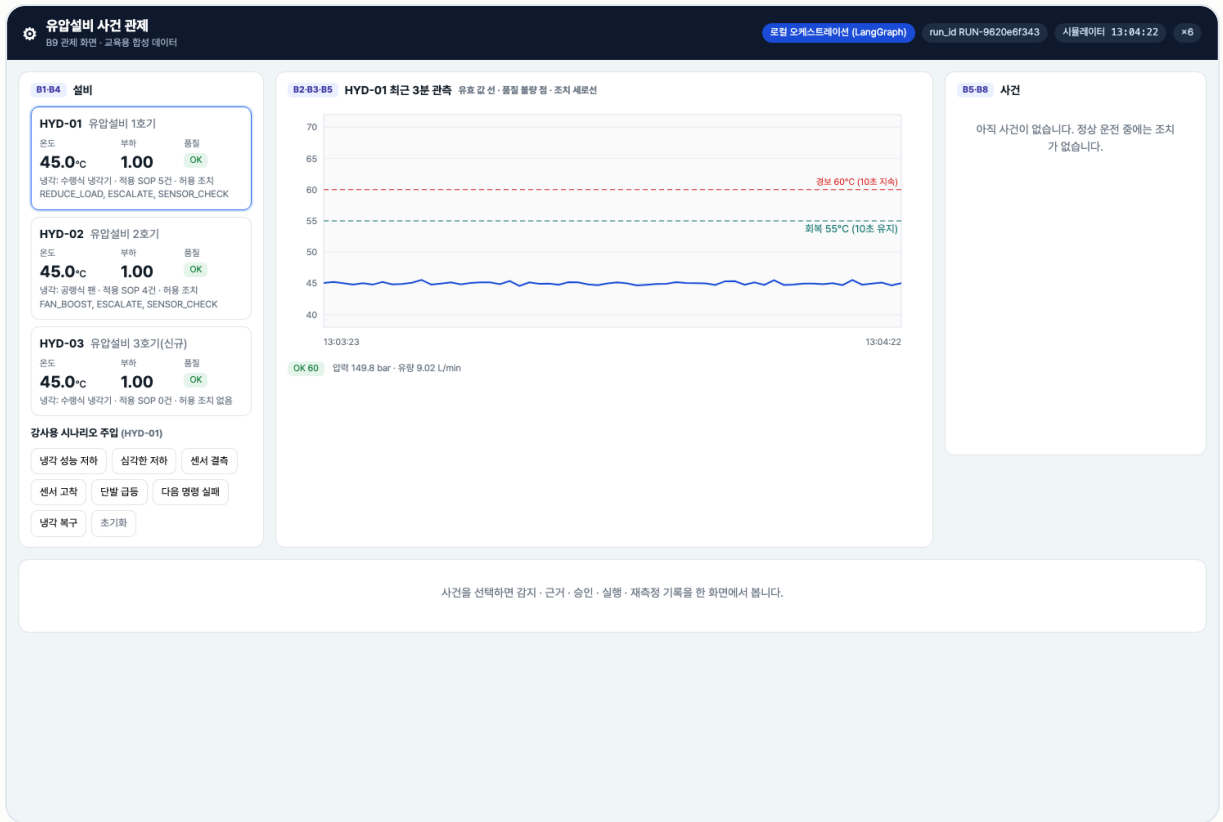
3. 활동 1 — 완성 시스템 시연과 아키텍처 읽기 · 50분

목표

완성 시스템이 실제로 돌아가는 모습을 보고, 화면의 각 영역이 어느 블록의 결과인지 대응시킨다.

따라 하기

1. 강사가 띄운 관제 화면 <http://localhost:8800> 을 연다. 서버 기동은 강사가 한다(학생은 `scripts/servers.sh` 를 실행하지 않는다).



관제 화면 정상 운전

2. 화면을 블록에 대응시킨다.

화면 영역	블록	오늘 연결
왼쪽 설비 카드(온도·부하·품질 OK, 적용 SOP 건수)	B1-B4	온도·품질 값은 B1→B2 결과
가운데 "HYD-01 최근 3분 관측" 그래프, OK 60 배지	B2-B3-B5	B3 최근 구간 조회 결과
오른쪽 사건 목록	B5-B8	5회차 이후
상단 run_id RUN-...	B3	오늘 쓰는 run 개념

3. 화면이 부르는 API 를 직접 호출한다(읽기 전용 GET).

```
curl -s http://localhost:8800/api/state | python3 -c "import json,sys;
d=json.load(sys.stdin); print(list(d.keys())); print(d['sim']['HYD-01'])"
```

2026-09-14 실행 결과(시각·값은 실행마다 다르다):

```
[ 'run_id', 'mode', 'speed', 'paused', 'sim', 'last_error' ]
{ 'asset_id': 'HYD-01', 't': '2026-09-14T15:13:43+00:00', 'temp_c':
54.000000000000004, 'load': 0.8, 'cooling_eff': 0.4, 'step_no': 7413,
```

```
'sensor_fault': None, 'sensor_fault_left': 0, 'stuck_value': None,
'command_fail_next': False, 'steady_temp_c': 54.0}
```

4. 화면 그래프가 읽는 관측은 PostgreSQL 에 있다. 가장 최근 관측 3행의 출처를 본다.

```
docker compose exec -T postgres psql -U hydops -d hydops -c "SELECT
r.source, o.sensor_id, o.unit, o.origin_cycle_id, o.elapsed_s,
o.is_synthetic, o.agg FROM observation o JOIN run r USING (run_id)
WHERE o.asset_id='HYD-01' ORDER BY o.ts DESC, o.sensor_id LIMIT 3;"
```

source	sensor_id	unit	origin_cycle_id	elapsed_s	is_synthetic
agg					
-----+-----+-----+-----+-----+-----					
simulator	FS1	L/min		7452	t
simulator	PS1	bar		7452	t
simulator	TS1	°C		7452	t
(3 rows)					

시연 서버의 관측은 시뮬레이터가 만든 합성 데이터다. `origin_cycle_id` 가 비어 있고 `is_synthetic` 이 `t` 다. 오늘 우리가 넣을 UCI 재생 관측은 반대로 채워져야 한다.

확인 포인트

- 화면의 "교육용 합성 데이터" 표시와 DB 의 `source=simulator` , `is_synthetic=t` 가 같은 사실을 가리킨다.
- 시뮬레이터 관측의 `elapsed_s` 는 사이클 위상이 아니라 실행 시작 후 경과 초(`step_no`)다.

흔한 실수

- 화면에 보이는 온도를 UCI 설비의 실측값으로 설명한다. 시연 온도는 `simulator.py` 의 교육용 가정 모델($T_{ss} = 30 + 15 \cdot \text{load}^2 / \text{cooling_eff}$)에서 나온다.
- 관제 화면의 초기화 버튼을 누른다. 수업 서버는 여러 사람이 함께 쓰므로 버튼 조작은 강사만 한다.

4. 활동 2 — 세 센서 CSV를 공통 관측 형식으로 매핑 · 50분

목표

labs/practical/S01/mapping.py 의 reduce_to_seconds 와 map_cycle 을 고쳐 세 센서를 공통 관측 레코드로 만든다.

따라 하기

1. 시작본의 축약 함수를 읽는다. 첫 샘플만 고르고 있다.

```
# lecture/labs/practical/S01/mapping.py (시작본 발췌)
def reduce_to_seconds(samples: np.ndarray, hz: int) -> list[dict]:
    """60초 × hz 샘플을 1초 단위 mean/min/max/n 60개로 축약한다."""
    if len(samples) != CYCLE_S * hz:
        raise ValueError(f"샘플 수 {len(samples)} != {CYCLE_S} × {hz}")
    shaped = samples.reshape(CYCLE_S, hz)
    # TODO (학생): 지금은 1초 구간의 첫 샘플만 골라 쓴다(숙아내기). ...
    return [{"mean": float(sec[0]), "min": float(sec[0]), "max":
            float(sec[0]), "n": 1} for sec in shaped]
```

2. 사이클 #100 의 PS1 첫 1초를 시작본(st)과 정답본(sol) 함수로 비교한다. 두 모듈은 lecture/system 에서 PYTHONPATH=. .venv/bin/python 으로 연 파이썬에서 다음처럼 불러온다(test_check.py 와 같은 방식).

```
import importlib.util
def load(path, name):
    spec = importlib.util.spec_from_file_location(name, path)
    mod = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(mod); return mod
st = load("../labs/practical/S01/mapping.py", "st") # 시작본 (내가 고치는 파일)
sol = load("../labs/practical/S01/solution/mapping.py", "sol") # 정답본
```

```
raw = sol.read_raw_cycle("PS1", 100)
print("PS1 samples", raw.shape, raw[:5])
print("starter sec0", st.reduce_to_seconds(raw, 100)[0])
print("solution sec0", sol.reduce_to_seconds(raw, 100)[0])
```

```
PS1 samples (6000,) [147.21 147.3 147.38 146.64 143.59]
starter sec0 {'mean': 147.21, 'min': 147.21, 'max': 147.21, 'n': 1}
```

```
solution sec0 {'mean': 170.16209999999998, 'min': 141.27, 'max': 189.88, 'n': 100}
```

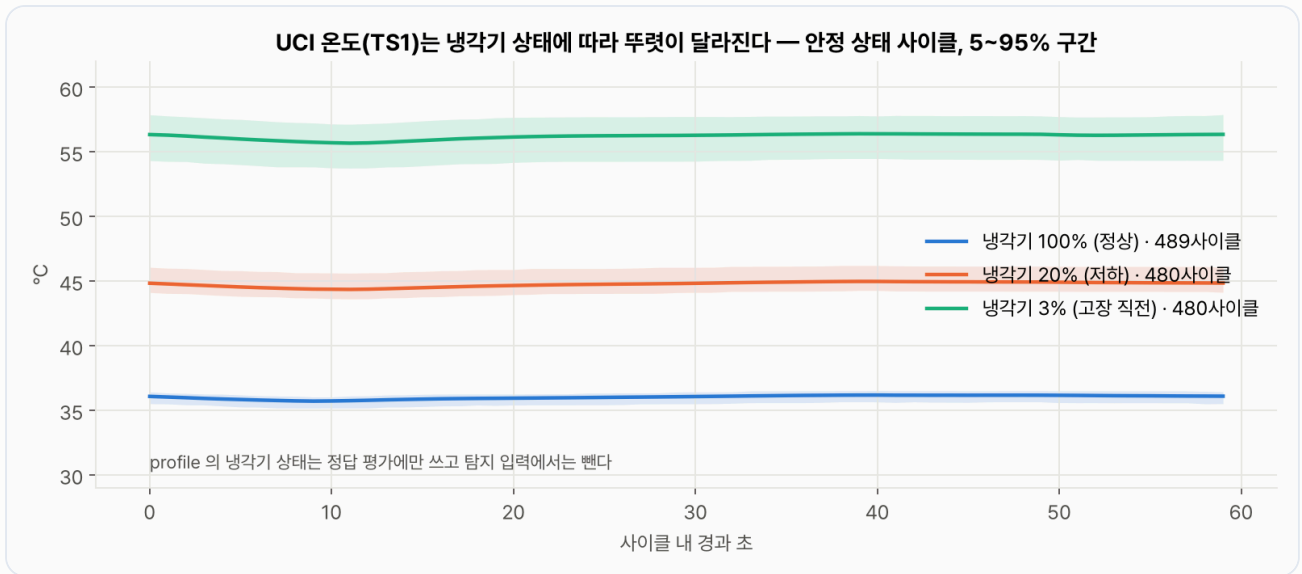
첫 1초 안에서 압력이 141.27~189.88 bar 사이를 오가므로, 첫 샘플 147.21 은 그 1초를 대표하지 못한다.

3. `reduce_to_seconds` 를 `shaped` 의 행마다 `mean()` · `min()` · `max()` · `size` 로 고친다.
4. `map_cycle` 의 TODO 네 곳을 고친다: `origin_cycle_id=int(cycle_id)` , `unit=spec["unit"]` , 1 Hz 가 아니면 `agg={"min","max","n","source_hz"}` , `is_synthetic=False` .
5. 완성 시스템의 매핑과 비교해 본다. 강사 제공 축약본(`data/reduced/hydraulic_1s.npz`) 으로 만든 레코드다.

```
PYTHONPATH=. .venv/bin/python -c "  
from datetime import datetime, timezone  
from hydops.b1_data import uci  
ds = uci.load_reduced()  
rows = uci.cycle_observations(ds, 100, 'HYD-01',  
                             datetime(2026,10,31,14,tzinfo=timezone.utc))  
print(len(rows))  
for sid in ('TS1','PS1','FS1'):  
    print([x for x in rows if x['sensor_id']==sid][0])  
print(ds.profile.loc[100].to_dict())  
"
```

```
180  
{'asset_id': 'HYD-01', 'sensor_id': 'TS1', 'ts': datetime.datetime(2026, 10,  
31, 14, 0, tzinfo=datetime.timezone.utc), 'elapsed_s': 0, 'origin_cycle_id':  
100, 'raw_value': 53.219, 'unit': '°C', 'agg': None, 'is_synthetic': False}  
{'asset_id': 'HYD-01', 'sensor_id': 'PS1', 'ts': datetime.datetime(2026, 10,  
31, 14, 0, tzinfo=datetime.timezone.utc), 'elapsed_s': 0, 'origin_cycle_id':  
100, 'raw_value': 170.16209999999995, 'unit': 'bar', 'agg': {'min': 141.27,  
'max': 189.88, 'n': 100, 'source_hz': 100}, 'is_synthetic': False}  
{'asset_id': 'HYD-01', 'sensor_id': 'FS1', 'ts': datetime.datetime(2026, 10,  
31, 14, 0, tzinfo=datetime.timezone.utc), 'elapsed_s': 0, 'origin_cycle_id':  
100, 'raw_value': 0.9027999999999997, 'unit': 'L/min', 'agg': {'min': 0.0,  
'max': 7.653, 'n': 10, 'source_hz': 10}, 'is_synthetic': False}  
{'cooler_pct': 3, 'valve_pct': 100, 'pump_leak': 0, 'accumulator_bar': 130,  
'stable_flag': 1, 'is_stable': False}
```

사이클 #100 은 profile 상 냉각기 3% · 안정 상태 아님(`stable_flag 1`)이다. 온도가 53°C 대인 이유다. 이 라벨은 레코드에 넣지 않는다.



냉각기 상태별 온도

확인 포인트

- 한 사이클 = 3센서 × 60초 = 180 레코드.
- 고친 `reduce_to_seconds` 의 PS1 평균이 `hydraulic_1s.npz` 의 평균과 오차 $1e-6$ 안에서 같다(S01 테스트 `test_reduce_keeps_mean_min_max_n`).
- 레코드에 `cooler_pct` 같은 정답 라벨 키가 없다.

흔한 실수

- 단위를 온도 기준 하나로 통일한다. 압력 bar 와 유량 L/min 이 °C 로 저장되면 뒤 회차 그래프 결합에서 단위 불일치를 찾을 수 없다.
- 탐지 성능을 올리려고 `cooler_pct` 를 관측 레코드에 넣는다. 정답 라벨은 4회차 평가에만 쓴다.

5. 활동 3 — 제공 적재 함수를 PostgreSQL에 연결 · 50분

목표

매핑한 레코드를 제공 품질 검사기(B2)와 제공 적재 함수(B3)에 연결해 `LAB-run` 으로 적재한다.

따라 하기

1. 제공 적재 함수를 읽는다. COPY 로 한 번에 넣고, 레코드에 `value` · `quality_flag` 가 없으면 NULL · 'OK' 로 들어간다. 그래서 품질 검사를 먼저 거쳐야 한다.

```
# lecture/system/hydops/b3_tsdb/store.py (발췌)
def insert_observations(run_id: str, rows: list[dict]) -> int:
    ...
    with cur.copy(
        "COPY observation (run_id, asset_id, sensor_id, ts, elapsed_s,
        origin_cycle_id, raw_value, value, unit, quality_flag, agg,
        is_synthetic) FROM STDIN"
    ) as cp:
        for r in rows:
            cp.write_row((run_id, r["asset_id"], r["sensor_id"],
            r["ts"], r.get("elapsed_s"), r.get("origin_cycle_id"),
            r.get("raw_value"), r.get("value"), r["unit"],
            r.get("quality_flag", "OK"),
            json.dumps(r["agg"]) if r.get("agg") else
            None, r.get("is_synthetic", False)))
```

2. 시작본 `load_cycles` 의 TODO 두 곳을 고친다.

```
# lecture/labs/practical/S01/solution/mapping.py (정답본 발췌)
store.create_run("uci replay", "lab-s01", note="UCI 원본 재생 (ts 는 재생 시각)",
    run_id=run_id)
checker = SensorQualityChecker()
total = 0
for i, cid in enumerate(cycle_ids):
    raw = {sid: read_raw_cycle(sid, cid) for sid in SENSOR_SPEC}
    rows = map_cycle(raw, cid, asset_id, start + timedelta(seconds=i *
        CYCLE_S))
    total += store.insert_observations(run_id, checker.check_many(rows))
```

시작본은 `create_run("simulator", ...)` 로 원본을 합성이라 기록하고, 모든 사이클을 같은 `start` 에서 재생해 시각이 겹친다.

3. 세 사이클을 적재한다. `run_id` 는 반드시 `LAB-` 로 시작하고 자기 이름을 붙인다.

```
n = sol.load_cycles("LAB-P01-demo", [100, 101, 102], "HYD-01")
print("inserted", n)
```

```
inserted 540
```

4. 적재한 run 과 품질 플래그 분포를 psql 로 확인한다.

```
docker compose exec -T postgres psql -U hydops -d hydops \
-c "SELECT run_id, source, scenario, note FROM run WHERE run_id='LAB-P01-
demo'" \
-c "SELECT quality_flag, count(*) FROM observation WHERE run_id='LAB-P01-
demo' GROUP BY 1"
```

run_id	source	scenario	note
LAB-P01-demo	uci_replay	lab-s01	UCI 원본 재생 (ts 는 재생 시각)

(1 row)

quality_flag	count
OK	540

(1 row)

5. 품질이 모두 OK 인 것이 당연한지 확인한다. 유량 FS1 은 사이클 안에서 크게 뛴다. 사이클 101 의 앞 12초를 본다.

```
ck = SensorQualityChecker(); out = []
for i, c in enumerate([100, 101, 102]):
    raw = {s: sol.read_raw_cycle(s, c) for s in sol.SENSOR_SPEC}
    out += ck.check_many(sol.map_cycle(raw, c, "HYD-01",
    sol.LAB_REPLAY_START + timedelta(seconds=60 * i)))
print("non-OK", sum(r["quality_flag"] != "OK" for r in out))
print([round(r["raw_value"], 2) for r in out if r["sensor_id"] == "FS1" and
r["origin_cycle_id"] == 101][:12])
```

```
non-OK 0
[0.91, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 8.94, 7.78, 7.75]
```

유량이 9초째에 0 에서 8.94 L/min 으로 뛰는데도 SPIKE 가 아니다. 급등 기준이 **정상 운전의 실측 변화폭**에서 정해졌기 때문이다. 제공 설정의 주석과 기준값이다.

```
# lecture/system/hydops/config.py (발췌)
# UCI 정상 사이클의 1초 최대 변화: TS1 0.23°C · PS1 40.3 bar · FS1 10.9 L/min (펌프
기동) → 그보다 큰 점프만 급등
spike_delta: dict = field(default_factory=lambda: {"TS1": 8.0, "PS1": 60.0,
"FS1": 15.0})
```

주석의 수치를 축약본 2,205 사이클 전체에서 직접 계산해 확인한다(사이클 안의 이웃한 1초 평균 끼리 차이).

```
PYTHONPATH=., .venv/bin/python -c "  
import numpy as np  
from hydops.b1_data import uci  
ds = uci.load_reduced()  
print({s: round(float(np.abs(np.diff(ds.sensors[s]['mean'], axis=1)).max()),  
                2) for s in ('TS1', 'PS1', 'FS1')})  
"
```

```
{'TS1': 0.23, 'PS1': 40.26, 'FS1': 10.94}
```

FS1의 정상 최대 변화 10.94 L/min(펌프 기동)보다 큰 15.0을 기준으로 두었으므로, 운전 위상 변화는 급등으로 막히지 않는다. 기준을 문헌이나 감으로 정하면 이 점프를 센서 오류로 지우게 된다 — 실제로 이 시스템의 초기 설정(FS1 6 L/min)에서는 세 사이클 적재에서 FS1 SPIKE가 생겼고, 정상 사이클의 실측 변화폭을 재어 기준을 고쳤다. 품질 기준은 **정상 동작을 먼저 측정하고** 그 바깥으로 정한다.

확인 포인트

- `inserted 540` = 3 사이클 × 180.
- `run.source = uci_replay`.
- 모든 행에 `quality_flag`가 붙어 있고, 정상 UCI 사이클 세 개는 540행 모두 OK다.

흔한 실수

- 재생 시작 시각을 오늘 날짜 이후로 둔다. 수업 서버의 시뮬레이터는 "지금" 시각으로 관측을 쌓는데, 교육용 플랫폼의 `Asset.recent_state`처럼 `run_id` 없이 `max(ts)`로 설비 최신 구간을 읽는 조치가 있다. 미래 시각의 재생 관측이 들어가면 그 조치가 우리 재생 데이터를 "최신"으로 읽는다. 실습 파일의 `LAB_REPLAY_START`가 2026-09-01인 이유다.
- 실습 중 `store.init_schema(reset=True)`나 TRUNCATE로 테이블을 비운다. 공유 DB이므로 금지한다. 자기 행은 `delete_lab_run(run_id)`로 지운다.

6. 활동 4 — 최근 60초를 조회하고 원본 사이클 추적 · 50분

목표

제공 조회 함수 `recent_window` 로 최근 60초를 읽고, 센서별로 단위·원본 사이클·위상·원본/합성 구분을 요약한다.

따라 하기

1. 제공 조회 함수를 읽는다. 기준 시각은 해당 설비의 마지막 관측 시각이고, `run_id` 를 주면 그 run 안에서만 찾는다.

```
# lecture/system/hydops/b3_tsdb/store.py (발췌)
def recent_window(asset_id: str, seconds: int = 60, sensor_id: str | None =
    None, run_id: str | None = None, until: datetime | None = None) ->
    list[dict]:
    """설비별 최근 구간 SELECT. 기준 시각은 해당 설비의 마지막 관측 시각(재생 시각) 이다."""
    with connect() as c:
        if until is None:
            q = "SELECT max(ts) AS t FROM observation WHERE asset_id=%s" +
                (" AND run_id=%s" if run_id else "")
            until = c.execute(q, (asset_id, run_id) if run_id else
                (asset_id,)).fetchone()["t"]
```

2. 시작본 `window_with_origin` 은 `run_id` 를 넘기지 않는다. 적재해 둔 `LAB-P01-demo` 에 대해 실행한 결과다.

```
w3 = st.window_with_origin("LAB-P01-demo", "HYD-01", 60)
print("starter window", w3["from"], w3["to"], {k: (v["origin_cycles"],
    v["run_ids"]) for k, v in w3["sensors"].items()})
```

```
starter window 2026-09-14 14:25:55+00:00 2026-09-14 14:26:54+00:00 {'FS1':
([None], ['RUN-b5021e42b2']), 'PS1': ([None], ['RUN-b5021e42b2']), 'TS1':
([None], ['RUN-b5021e42b2'])}
```

우리 run 이 아니라 수업 서버 시뮬레이터 run 의 최근 60초가 나왔다. 원본 사이클은 `None` 이다.

3. `store.recent_window(asset_id, seconds, run_id=run_id, until=until)` 로 고치고 다시 실행한다.

```
w = sol.window_with_origin("LAB-P01-demo", "HYD-01", 60)
print("from", w["from"], "to", w["to"])
for sid, s in w["sensors"].items():
    print(sid, s["unit"], s["samples"], s["origin_cycles"],
          min(s["elapsed"]), max(s["elapsed"]), s["is_synthetic"],
          s["run_ids"])
```

```
from 2026-09-01 00:02:00+00:00 to 2026-09-01 00:02:59+00:00
FS1 L/min 60 [102] 0 59 [False] ['LAB-P01-demo']
PS1 bar 60 [102] 0 59 [False] ['LAB-P01-demo']
TS1 °C 60 [102] 0 59 [False] ['LAB-P01-demo']
```

4. SQL 로 같은 질문을 한다. 설비·run·최근 60초 조건과 원본 사이클 그룹을 모두 쓴다.

```
SELECT sensor_id, unit, origin_cycle_id, count(*) AS n, min(elapsed_s) AS
e0, max(elapsed_s) AS e1, bool_or(is_synthetic) AS synthetic
FROM observation WHERE run_id='LAB-P01-demo' AND asset_id='HYD-01'
AND ts > (SELECT max(ts) FROM observation WHERE run_id='LAB-P01-demo') -
interval '60 seconds'
GROUP BY sensor_id, unit, origin_cycle_id ORDER BY sensor_id;
```

sensor_id	unit	origin_cycle_id	n	e0	e1	synthetic
FS1	L/min	102	60	0	59	f
PS1	bar	102	60	0	59	f
TS1	°C	102	60	0	59	f

(3 rows)

한 행을 골라 레코드 전체를 본다.

sensor_id	ts	elapsed_s	origin_cycle_id	raw
FS1	2026-09-01 00:02:00+00	0	102	0.93
OK	{ "n": 10, "max": 7.834, "min": 0.003, "source_hz": 10 }			
PS1	2026-09-01 00:02:00+00	0	102	170.08
OK	{ "n": 100, "max": 189.84, "min": 141.47, "source_hz": 100 }			
TS1	2026-09-01 00:02:00+00	0	102	53.38
OK				

5. 사이클 경계에 걸친 60초를 조회한다. 기준 시각을 재생 시작 +90초로 준다.

```
w2 = sol.window_with_origin("LAB-P01-demo", "HYD-01", 60,
                           until=sol.LAB_REPLAY_START + timedelta(seconds=90))
print("mid", w2["from"], w2["to"], w2["sensors"]["TS1"]["origin_cycles"])
```

```
mid 2026-09-01 00:00:31+00:00 2026-09-01 00:01:30+00:00 [100, 101]
```

구간이 `ts > 기준-60초` 이므로 00:00:31(사이클 100 의 31초)부터 00:01:30(사이클 101 의 30초)까지다. 재생 시각은 연속이지만 원본은 두 사이클이다.

6. 실습을 마치면 자기 행을 지운다.

```
sol.delete_lab_run("LAB-P01-demo")
```

확인 포인트

- 한 행에서 설비 ID(`HYD-01`)·센서·단위·원본 사이클·위상·원본/합성 구분을 모두 읽을 수 있다 — 이것이 오늘의 완료 확인이다.
- `run_id` 조건이 없으면 다른 `run` 의 관측이 섞인다.

흔한 실수

- 최근 60초를 `ts >= 기준-60초` 로 써서 61개를 받는다. 제공 함수는 `ts > %s AND ts <= %s` 다.
- 사이클 경계 구간의 온도 변화를 설비 변화로 해석한다. 서로 다른 원본 사이클을 이어 붙인 경계일 수 있으므로 `origin_cycle_id` 를 먼저 본다.

7. 실습 과제

- 폴더: `labs/practical/S01/` — `TASK.md` , `mapping.py` (시작본), `solution/mapping.py` , `test_check.py`
- 수정할 함수(`# TODO(학생)` 9곳): `reduce_to_seconds` (1초 mean·min·max·n), `map_cycle` (원본 사이클·단위·agg·원본 구분), `load_cycles` (`run.source`, 사이클별 재생 시각), `window_with_origin` (`run_id` 조건)

- 검증 방법: 사이클 100 의 PS1 평균이 강사 축약본과 같은지, 180 레코드의 단위
·agg·origin_cycle_id 가 맞는지, LAB-S01- run 으로 사이클 100·101·102 를 적재했을 때 마
지막 60초가 사이클 102 의 0~59초이고 +90초 기준 60초가 사이클 100·101 인지 확인한 뒤 자
기 행을 지운다.
- 확인 명령:

```
cd lecture/system && PYTHONPATH=. .venv/bin/python -m pytest
../labs/practical/S01 -q
```

시작본 실행 결과(요약):

```
E   AssertionError: assert {'°C'} == {'bar'}
E   AssertionError: assert 'simulator' == 'uci_replay'
FAILED ../labs/practical/S01/test_check.py::test_reduce_keeps_mean_min_max_n
FAILED ../labs/practical/S01/test_check.py::test_map_cycle_common_record -
As...
FAILED
../labs/practical/S01/test_check.py::test_load_and_recent_window_tracks_origin
3 failed in 0.27s
```

모두 고친 뒤: 3 passed .

8. 완료 확인 체크리스트

- ☐ 관제 화면의 세 영역(설비·최근 관측·사건)을 블록 번호로 말할 수 있다.
- ☐ reduce_to_seconds 의 PS1 결과가 1초 mean·min·max 와 n=100 을 가진다.
- ☐ 한 사이클의 레코드가 180개이고, 단위가 TS1 °C · PS1 bar · FS1 L/min 이다.
- ☐ LAB- run 이 source=uci_replay 로 저장되었다.
- ☐ 최근 60초 조회 결과 한 행에서
asset_id · unit · origin_cycle_id · elapsed_s · is_synthetic 을 읽어 설명했
다.
- ☐ 사이클 경계에 걸친 구간에서 원본 사이클 두 개를 확인했다.
- ☐ pytest ../labs/practical/S01 -q 가 3 passed 다.
- ☐ 자기 LAB- 행을 지웠다.

9. 회고 (10분)

1. 재생 시각(`ts`)과 원본 사이클·위상(`origin_cycle_id · elapsed_s`)을 나눠 두지 않았다면, 4회차에 "사이클 102 의 12초째 온도가 정상 위상 기준보다 높다"는 판단을 할 수 있었을까?
2. PS1 1초 안에서 141~190 bar 로 흔들리는 값을 평균 하나로만 남기면 무엇을 잃는가? `agg` 의 min·max 는 어떤 질문에 쓰일까?
3. FS1 이 9초째 0 → 8.94 L/min 으로 오르는 점프는 SPIKE 가 아니다. 급등 기준을 정상 운전의 실측 변화폭(최대 10.94 L/min)에서 정하지 않고 6 L/min 처럼 정했다면 무엇이 지워졌을까?

개인 설명 과제 — 내가 설명할 질의 또는 실패 분기 하나: 팀원 각자 다음 중 하나를 골라 2분 안에 설명한다. (a) run_id 없는 `recent_window` 가 시뮬레이터 run 을 읽은 실패 분기, (b) 사이클 경계에 걸친 60초 SQL, (c) `source` CHECK 제약이 원본/합성 구분을 지키는 방식, (d) FS1 급등 기준을 정상 사이클의 1초 최대 변화에서 정한 근거.

강사 노트

시간 배분

시각	내용
14:00~14:50	활동 1 시연·아키텍처 읽기 (완성 시스템 시연 20분, 화면·블록 대응 표 작성 20분, API·DB 확인 10분)
14:50~15:40	활동 2 매핑 (첫 샘플 vs 평균 비교를 먼저 보여 준 뒤 수정)
15:40~16:10	휴식 30분
16:10~17:00	활동 3 적재
17:00~17:50	활동 4 조회·원본 추적, pytest
17:50~18:00	회고

사전 준비

- `docker compose up -d` 로 PostgreSQL·Neo4j 가 떠 있는지 확인한다. 스키마는 `store.init_schema()` (reset 없이) 로 한 번만 만든다.
- 관제 화면 시연은 `scripts/servers.sh local 4` 로 강사가 띄운다. 플랫폼 bootstrap 은 오늘 필요 없다.

- `data/reduced/hydraulic_1s.npz` 와 `profile.csv` 가 있는지 확인한다(없으면 `load_reduced()` 가 원본에서 만든다).
- 학생별 run_id 규칙(`LAB-<팀>--<이름>`)을 칠판에 적는다.

장애 대응

- 수업 서버가 멈추면 활동 1 은 스크린샷(S01)으로 대신하고, 활동 2~4 는 DB 만 있으면 진행된다.
- 학생이 run_id 없이 적재해 `RUN-...` 이 생기면, 강사가 해당 run_id 의 observation → run 순으로 지운다(다른 행은 건드리지 않는다).
- PS1 원본 파일(87MB) 한 줄 읽기가 느린 노트북에서는 사이클 번호를 작게 잡는다.

용어

용어	뜻
공통 관측 레코드	센서·주기가 달라도 같은 필드(asset_id, sensor_id, ts, elapsed_s, origin_cycle_id, raw_value, unit, agg, is_synthetic)를 갖는 1초 단위 행
재생 시각(replay ts)	UCI 사이클을 이어 붙여 재생할 때 붙인 시각. 실제 수집 시각이 아니다
origin_cycle_id	UCI 원본 사이클 번호. 합성 데이터는 NULL
elapsed_s	사이클 안의 경과 초(위상). 시뮬레이터는 실행 시작 후 경과 초
1초 축약	고주파 샘플을 1초 mean·min·max·n 으로 요약하는 것. 첫 샘플만 고르는 속아내기와 다르다
run	한 번의 적재·실행 단위. <code>source</code> 로 원본 재생(uci_replay)과 시뮬레이터(simulator)를 구분한다
정답 라벨	profile.txt 의 냉각기 상태 등. 평가에만 쓰고 탐지 입력에 넣지 않는다
LAB- run	실습용 run_id 접두사. 끝나면 자기 행만 지운다